# Diseases, Models, and How to Extend the System

## Summary

This document explains:

- Which diseases the project currently supports
- How the system performs diagnosis (high-level architecture)
- The role of local AI models (where they live, how they are used)
- Whether models are pre-trained and how to train or fine-tune them locally
- Step-by-step instructions to add or remove a disease and integrate a model
- Validation, evaluation, and deployment notes

---

## 1) Diseases currently supported

Based on the code and configuration in this repository, the project supports (or provides frameworks for):

- Brain tumor detection / segmentation (MRI) — framework/placeholder present
- Chest X-ray disease classification (COVID-19 / pneumonia screening)
- Skin lesion classification & segmentation (melanoma vs benign)

Notes:

- The chest X-ray and skin lesion agents have concrete inference modules under `agents/image_analysis_agent/`.
- The brain tumor pipeline is scaffolded but may still require the model artifact to be added at `agents/image_analysis_agent/brain_tumor_agent/models/`.

---

## 2) How diagnosis is performed (architecture)

High-level flow for an image-based diagnosis:

1. User uploads an image via the frontend.
2. `app.py` receives the upload and forwards it to the agent decision pipeline (`agents.agent_decision.process_query`).
3. `AgentDecision` routes the request to `ImageAnalysisAgent` (see `agents/image_analysis_agent/__init__.py`).

4. `ImageAnalysisAgent` runs a fast image-type classifier (`ImageClassifier`) to detect whether the image is a chest X-ray, skin lesion, MRI, or non-medical.
5. The image is routed to the task-specific inference module:
    - Chest X-ray: `chest_xray_agent` inference class
    - Skin lesion: `skin_lesion_agent` inference class
    - Brain tumor: `brain_tumor_agent` inference class (if present)
6. The inference module returns labels, confidence scores, and (if implemented) segmentation masks or visualization images saved under `uploads/skin_lesion_output/`.
7. If configured, responses that need verification are flagged for human-in-the-loop validation (see `agents/README.md` and `agent_decision.py`).

Non-image medical queries use the RAG agent (`agents/rag_agent`) which retrieves medical knowledge from the Qdrant vectorstore and returns context-grounded answers.

---

## 3) Role of local AI models

- Models are local artifacts stored in the repository (or referenced paths) and loaded by the inference wrappers. The model paths are configured in `config.py` under `MedicalCVConfig`:
    - `brain_tumor_model_path` (e.g. `./agents/image_analysis_agent/brain_tumor_agent/models/brain_tumor_segmentation.pth`)
    - `chest_xray_model_path` (e.g. `./agents/image_analysis_agent/chest_xray_agent/models/covid_chest_xray_model.pth`)
    - `skin_lesion_model_path` (e.g. `./agents/image_analysis_agent/skin_lesion_agent/models/checkpointN25_.pth.tar`)
- The local models run inference inside the respective agent modules (PyTorch-based). The `ImageAnalysisAgent` instantiates and calls these inference wrappers.
- Local models are responsible for the heavy-lifting of medical image interpretation (classification, segmentation). The LLMs are used for routing, explanation, post-processing explanations, and RAG-based textual answers.

---

## 4) Are the local models trained?

- The repository includes inference wrappers and references to model artifacts, but whether a model is "trained" depends on whether the checkpoint files exist in the model path.
- Check for presence of model files:
  - `agents/image_analysis_agent/chest_xray_agent/models/covid_chest_xray_model.pth`
  - `agents/image_analysis_agent/skin_lesion_agent/models/checkpointN25_.pth.tar`
  - `agents/image_analysis_agent/brain_tumor_agent/models/brain_tumor_segmentation.pth`
- If the artifact is present, it indicates a pre-trained model is included. If not present, you must obtain or train the model and place the checkpoint at the configured path.
- The repo does not include full training pipelines for every model by default. You will need training scripts or adapt public training code to create model checkpoints.

---

## 5) How to add a new disease (step-by-step)

This covers adding a new image-based disease (e.g., diabetic retinopathy from fundus images).

1. Data & Dataset
   - Collect a labeled dataset for your disease (images + labels, optionally segmentation masks).
   - Organize into `data/<disease_name>/train`, `data/<disease_name>/val`, `data/<disease_name>/test`.
2. Train or obtain a model
   - Use PyTorch training code (recommended). You can create `agents/image_analysis_agent/<disease>_agent/train.py` or reuse existing training scripts.
   - Typical training steps:

```
# create and activate venv/conda
pip install -r requirements.txt
python agents/image_analysis_agent/<disease>_agent/train.py --
```

- Export the final model checkpoint to a stable filename and path.

3. Implement an inference wrapper

- Create a new module: `agents/image_analysis_agent/<disease>_agent/<disease>_inference.py` implementing a class `DiseaseAgent` with a `predict(image_path)` method that returns `{'label':..., 'confidence':..., 'output_image': 'path/to/vis.png'}`.
- Follow style of `chest_xray_agent` and `skin_lesion_agent` for consistent API.

4. Register in `ImageAnalysisAgent`
   - Open `agents/image_analysis_agent/__init__.py` and:
     - import your new agent class
     - instantiate it in `ImageAnalysisAgent.__init__` using a path from `config.medical_cv` (add a new property if needed)
     - add a method on `ImageAnalysisAgent` like `def analyze_<disease>(self, image_path: str)` that forwards to the new agent.
5. Update configuration
   - Add model path(s) in `config.py` under `MedicalCVConfig`, e.g. `self.<disease>_model_path = "./agents/image_analysis_agent/<disease>_agent/models/model_final.pth"`.
6. Update `AgentDecision` routing
   - Modify `agents/agent_decision.py` routing logic so the `DECISION_SYSTEM_PROMPT` and routing rules send images of the new type to the new `analyze_<disease>` method. Update any JSON schema expectations if needed.
7. Frontend changes
   - If necessary, update the UI to display the new output image or to present disease-specific result fields.
8. Human-in-the-loop & Guardrails
   - If the disease requires human validation, add the appropriate flags in the response (e.g., `needs_validation=True`) and wire the `/validate` endpoint to accept clinician feedback.
9. Test end-to-end
   - Upload sample images from `sample_images/<disease>` and verify prediction, visualizations, and human validation flow.
10. Document & Version

- Update `PROJECT_CAPABILITIES.md` and `agents/README.md` to include the new disease and any special notes.

## 6) How to remove a disease

1. Remove model files from `agents/image_analysis_agent/<disease>_agent/models/`.
2. Remove or archive the agent folder `agents/image_analysis_agent/<disease>_agent/`.
3. Remove references in `agents/image_analysis_agent/__init__.py` and `config.py`.
4. Remove routing entries from `agents/agent_decision.py` and any frontend UI elements.
5. Run integration tests and ensure no import errors remain.
6. Commit and push changes.

Note: Prefer deprecation (marking code as removed) and follow a removal PR so you can rollback if needed.

## 7) Training tips & best practices

- Use transfer learning and pre-trained backbones (ResNet, EfficientNet, UNet) to reduce training time and improve performance.
- Normalize image sizes and apply domain-appropriate augmentations (rotation, brightness, CLAHE for medical images where appropriate).
- Keep a held-out test set, and track metrics: accuracy, precision, recall, F1, AUC, and (for segmentation) IoU / Dice coefficient.
- Use mixed-precision training (AMP) to speed up on modern GPUs.
- Keep model checkpoints and training logs (TensorBoard) under `runs/` or a model registry.

## 8) Evaluation & Clinical Validation

- Before deploying predictions to users, conduct clinical validation with domain experts.
- Run prospective evaluation on new clinical data; compare model outputs against clinician labels.
- Record failure cases and add them to the training set for iterative improvement.

## 9) Safety, Regulatory, and Guardrails

- The system includes guardrails (see `agents/guardrails/local_guardrails.py`) for content filtering and rules.
- Always add `needs_validation=True` for riskier diagnoses or low-confidence predictions.
- Add clear disclaimers in the UI: this is an AI-assist tool and not a substitute for professional diagnosis.
- Keep audit logs for predictions, model versions, and clinician validations.

---

## 10) Example: Add "Diabetic Retinopathy" quick checklist

- Prepare dataset under `data/diabetic_retinopathy/`
- Add training script `agents/image_analysis_agent/diabetic_retinopathy_agent/train.py`
- Train and export model to `agents/image_analysis_agent/diabetic_retinopathy_agent/models/dr_model.pth`
- Implement `agents/image_analysis_agent/diabetic_retinopathy_agent/inference.py` with `predict()`
- Update `agents/image_analysis_agent/__init__.py` and `config.py` for new model path
- Update routing in `agents/agent_decision.py`
- Add sample images to `sample_images/diabetic_retinopathy/`
- Test upload, prediction and optional human validation

---

## 11) Notes about RAG and non-image diseases

- For non-image disease support (textual diagnosis, guidelines, drug info), rely on the RAG pipeline (`agents/rag_agent`) by ingesting domain documents into the Qdrant store via `ingest_rag_data.py`.
- To add clinical docs for a disease, place PDFs under `data/raw/` and run:

```
python ingest_rag_data.py --dir ./data/raw
```

- The RAG agent will then be able to provide source-attributed answers about that disease.

---

## 12) Where to look in the codebase (quick links)

- `agents/image_analysis_agent/` — image classifiers & inference wrappers
- `agents/agent_decision.py` — routing and agent orchestration
- `agents/rag_agent/` — RAG ingestion & response generation
- `agents/guardrails/local_guardrails.py` — safety rules
- `config.py` — model path and LLM configuration
- `app.py` — FastAPI endpoints handling uploads, chat and validation

---

If you want, I can:

- add a template training script under `agents/image_analysis_agent/_templates/train_template.py`;
- add a small example that adds one sample disease end-to-end (train stub + inference + routing);
- or walk you through training a specific model on your machine.

Tell me which of these you'd like next.